

Практическая работа 5.

Процедура обратного распространения

ВВЕДЕНИЕ В ПРОЦЕДУРУ ОБРАТНОГО РАСПРОСТРАНЕНИЯ

Долгое время не было теоретически обоснованного алгоритма для обучения многослойных искусственных нейронных сетей. А так как возможности представления с помощью однослойных нейронных сетей оказались весьма ограниченными, то и вся область в целом пришла в упадок.

Разработка алгоритма обратного распространения сыграла важную роль в возрождении интереса к искусственным нейронным сетям. Обратное распространение – это систематический метод для обучения многослойных искусственных нейронных сетей. Он имеет солидное математическое обоснование. Несмотря на некоторые ограничения, процедура обратного распространения сильно расширила область проблем, в которых могут быть использованы искусственные нейронные сети, и убедительно продемонстрировала свою мощь.

Интересна история разработки процедуры. В [7] было дано ясное и полное описание процедуры. Но как только эта работа была опубликована, оказалось, что она была предвосхищена в [4]. А вскоре выяснилось, что еще раньше метод был описан в [12]. Авторы работы [7] сэкономили бы свои усилия, зная о работе [12]. Хотя подобное дублирование является обычным явлением для каждой научной области, в искусственных нейронных сетях положение с этим намного серьезнее из-за пограничного характера самого предмета исследования. Исследования по нейронным сетям публикуются в столь различных книгах и журналах, что даже самому квалифицированному исследователю требуются значительные усилия, чтобы быть осведомленным о всех важных работах в этой области.

ОБУЧАЮЩИЙ АЛГОРИТМ ОБРАТНОГО РАСПРОСТРАНЕНИЯ

Сетевые конфигурации

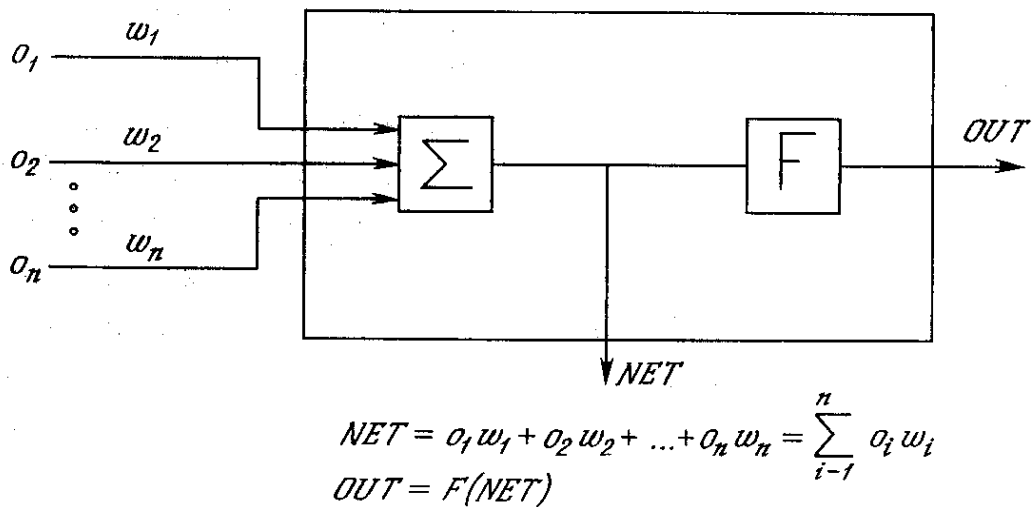


Рис. 3.1. Искусственный нейрон с активационной функцией

Нейрон. На рис. 3.1 показан нейрон, используемый в качестве основного строительного блока в сетях обратного распространения. Подается множество входов, идущих либо извне, либо от предшествующего слоя. Каждый из них умножается на вес, и произведения суммируются. Эта сумма, обозначаемая NET, должна быть вычислена для каждого нейрона сети. После того, как величина NET вычислена, она модифицируется с помощью активационной функции и получается сигнал OUT.

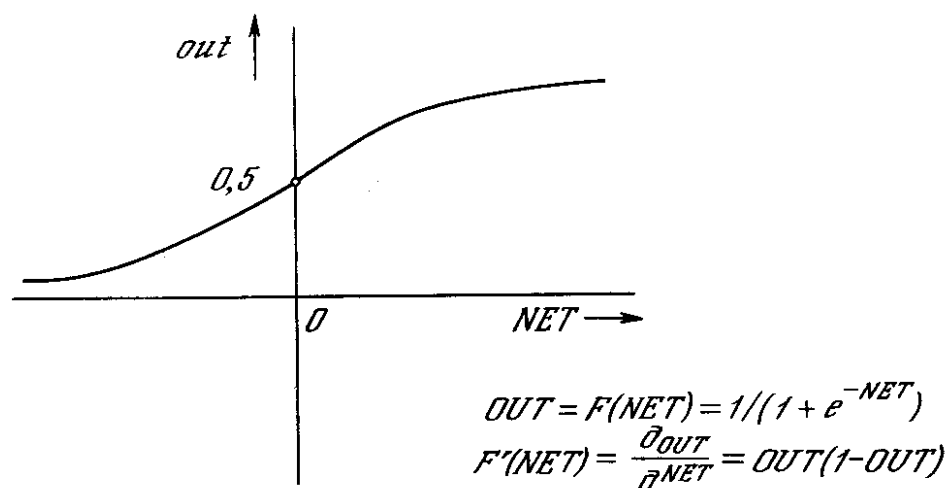


Рис. 3.2. Сигмоидальная активационная функция.

На рис. 3.2 показана активационная функция, обычно используемая для обратного распространения.

$$OUT = \frac{1}{1 + e^{-NET}}. \quad (3.1)$$

Как показывает уравнение (3.2), эта функция, называемая *сигмоидом*, весьма удобна, так как имеет простую производную, что используется при реализации алгоритма обратного распространения.

$$\frac{\partial OUT}{\partial NET} = OUT(1 - OUT). \quad (3.2)$$

Сигмоид, который иногда называется также *логистической*, или *сжимающей функцией*, сужает диапазон изменения NET так, что значение OUT лежит между нулем и единицей. Как указывалось выше, многослойные нейронные сети обладают большей представляющей мощностью, чем однослойные, только в случае присутствия нелинейности. Сжимающая функция обеспечивает требуемую нелинейность.

В действительности имеется множество функций, которые могли бы быть использованы. Для алгоритма обратного распространения требуется лишь, чтобы функция была всюду дифференцируема. Сигмоид удовлетворяет этому требованию. Его дополнительное преимущество состоит в автоматическом контроле усиления. Для слабых сигналов (величина NET близка к нулю) кривая вход-выход имеет сильный наклон, дающий большое усиление. Когда величина сигнала становится больше, усиление падает. Таким образом, большие сигналы воспринимаются сетью без насыщения, а слабые сигналы проходят по сети без чрезмерного ослабления.

Многослойная сеть.

На рис. 3.3 изображена многослойная сеть, которая может обучаться с помощью процедуры обратного распространения. (Для ясности рисунок упрощен.) Первый слой нейронов (соединенный с входами) служит лишь в качестве распределительных точек, суммирования входов здесь не производится. Входной сигнал просто проходит через них к весам на их выходах. А каждый нейрон последующих слоев выдает сигналы NET и OUT, как описано выше.

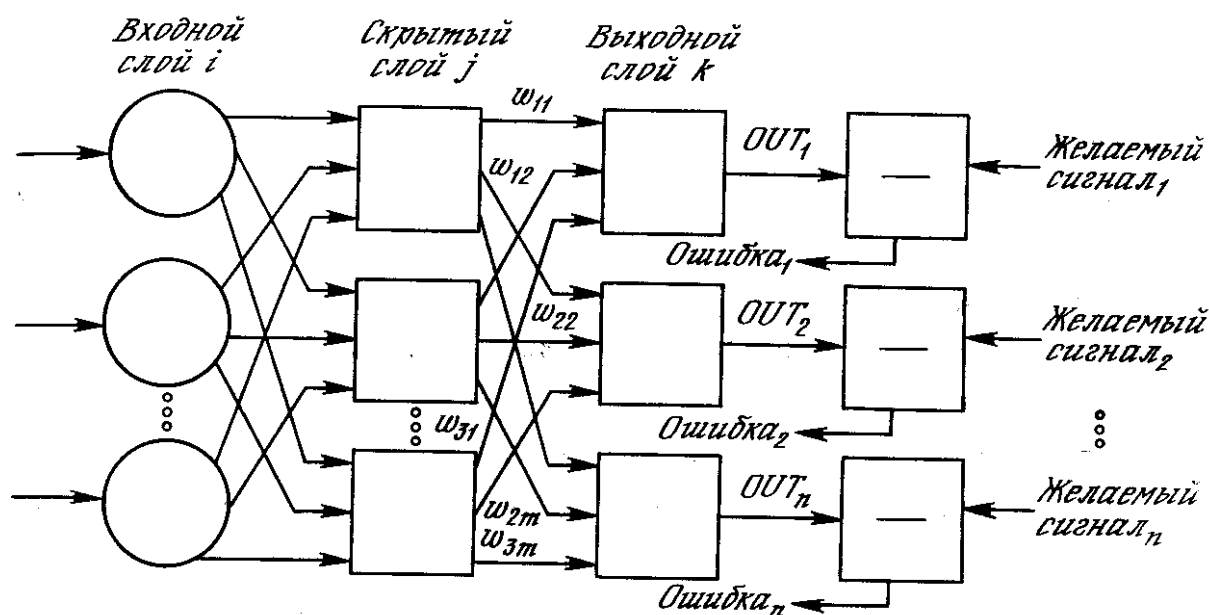


Рис. 3.3. Двухслойная сеть обратного распространения (ε – желаемый сигнал).

В литературе нет единообразия относительно того, как считать число слоев в таких сетях. Одни авторы используют число слоев нейронов (включая несуммирующий входной слой), другие – число слоев весов. Так как последнее определение функционально описательное, то оно будет использоваться на протяжении книги. Согласно этому определению, сеть на рис. 3.3 рассматривается как двухслойная. Нейрон объединен с множеством весов, присоединенных к его входу. Таким образом, веса первого слоя оканчиваются на нейронах первого слоя. Вход распределительного слоя считается нулевым слоем.

Процедура обратного распространения применима к сетям с любым числом слоев. Однако для того, чтобы продемонстрировать алгоритм, достаточно двух слоев. Сейчас будут рассматриваться лишь сети прямого действия, хотя обратное распространение применимо и к сетям с обратными связями. Эти случаи будут рассмотрены в данной главе позднее.

ОБЗОР ОБУЧЕНИЯ

Целью обучения сети является такая подстройка ее весов, чтобы приложение некоторого множества входов приводило к требуемому множеству выходов. Для краткости эти множества входов и выходов будут называться *векторами*. При обучении предполагается, что для каждого входного вектора существует парный ему целевой вектор, задающий требуемый выход. Вместе они называются *обучающей парой*. Как правило, сеть обучается на многих парах. Например, входная часть обучающей пары может состоять из набора нулей и единиц, представляющего

двоичный образ некоторой буквы алфавита. На рис. 3.4 показано множество входов для буквы «А», нанесенной на сетке. Если через квадрат проходит линия, то соответствующий нейронный вход равен единице, в противном случае он равен нулю. Выход может быть числом, представляющим букву «А», или другим набором из нулей и единиц, который может быть использован для получения выходного образа. При необходимости распознавать с помощью сети все буквы алфавита, потребовалось бы 26 обучающих пар. Такая группа обучающих пар называется *обучающим множеством*.

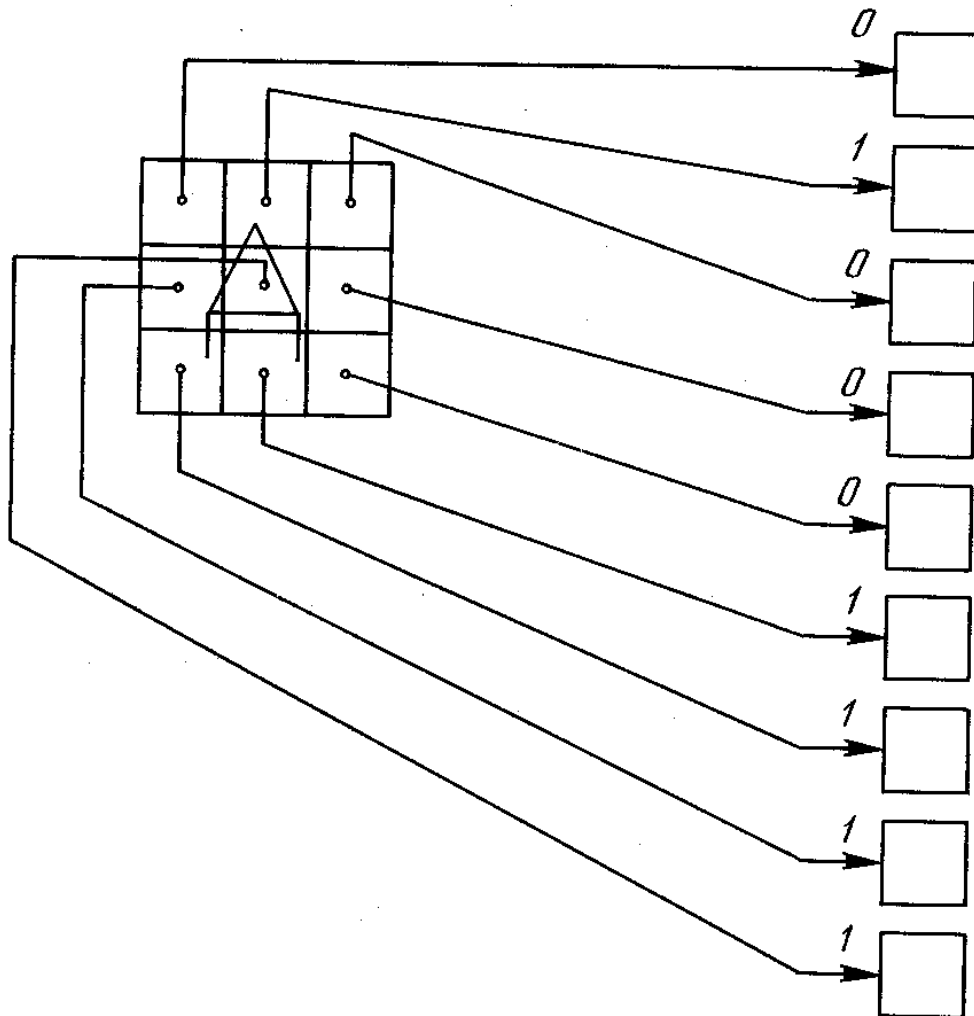


Рис. 3.4. Распознавание изображений

Перед началом обучения всем весам должны быть присвоены небольшие начальные значения, выбранные случайным образом. Это гарантирует, что в сети не произойдет насыщения большими значениями весов, и предотвращает ряд других патологических случаев. Например, если всем весам придать одинаковые начальные значения, а для требуемого функционирования нужны неравные значения, то сеть не сможет обучиться.

Обучение сети обратного распространения требует выполнения следующих операций:

1. Выбрать очередную обучающую пару из обучающего множества; подать входной вектор на вход сети.
2. Вычислить выход сети.
3. Вычислить разность между выходом сети и требуемым выходом (целевым вектором обучающей пары).
4. Подкорректировать веса сети так, чтобы минимизировать ошибку.
5. Повторять шаги с 1 по 4 для каждого вектора обучающего множества до тех пор, пока ошибка на всем множестве не достигнет приемлемого уровня.

Операции, выполняемые шагами 1 и 2, сходны с теми, которые выполняются при функционировании уже обученной сети, т.е. подается входной вектор и вычисляется получающийся выход. Вычисления выполняются послойно. На рис. 3.3 сначала вычисляются выходы нейронов слоя j , затем они используются в качестве входов слоя k , вычисляются выходы нейронов слоя k , которые и образуют выходной вектор сети.

На шаге 3 каждый из выходов сети, которые на рис. 3.3 обозначены OUT, вычитается из соответствующей компоненты целевого вектора, чтобы получить ошибку. Эта ошибка используется на шаге 4 для коррекции весов сети, причем знак и величина изменений весов определяются алгоритмом обучения (см. ниже).

После достаточного числа повторений этих четырех шагов разность между действительными выходами и целевыми выходами должна уменьшиться до приемлемой величины, при этом говорят, что сеть обучилась. Теперь сеть используется для распознавания и веса не изменяются.

На шаги 1 и 2 можно смотреть как на «проход вперед», так как сигнал распространяется по сети от входа к выходу. Шаги 3, 4 составляют «обратный проход», здесь вычисляемый сигнал ошибки распространяется обратно по сети и используется для подстройки весов. Эти два прохода теперь будут детализированы и выражены в более математической форме.

Проход вперед. Шаги 1 и 2 могут быть выражены в векторной форме следующим образом: подается входной вектор $\mathbf{X}$ и на выходе получается вектор $\mathbf{Y}$. Векторная пара вход-цель $\mathbf{X}$ и $\mathbf{T}$ берется из обучающего множества. Вычисления проводятся над вектором $\mathbf{X}$, чтобы получить выходной вектор $\mathbf{Y}$.

Как мы видели, вычисления в многослойных сетях выполняются слой за слоем, начиная с ближайшего к входу слоя. Величина NET каждого нейрона первого слоя вычисляется как взвешенная сумма входов нейрона. Затем активационная функция F «сжимает» NET и дает величину OUT для каждого нейрона в этом слое. Когда

множество выходов слоя получено, оно является входным множеством для следующего слоя. Процесс повторяется слой за слоем, пока не будет получено заключительное множество выходов сети.

Этот процесс может быть выражен в сжатой форме с помощью векторной нотации. Веса между нейронами могут рассматриваться как матрица $\mathbf{W}$. Например, вес от нейрона 8 в слое 2 к нейрону 5 слоя 3 обозначается $w_{8,5}$. Тогда NET-вектор слоя $\mathbf{N}$ может быть выражен не как сумма произведений, а как произведение $\mathbf{X}$ и $\mathbf{W}$. В векторном обозначении $\mathbf{N} = \mathbf{XW}$. Покомпонентным применением функции F к NET-вектору $\mathbf{N}$ получается выходной вектор $\mathbf{O}$. Таким образом, для данного слоя вычислительный процесс описывается следующим выражением:

$$\mathbf{O} = F(\mathbf{XW}). \quad (3.3)$$

Выходной вектор одного слоя является входным вектором для следующего, поэтому вычисление выходов последнего слоя требует применения уравнения (3.3) к каждому слою от входа сети к ее выходу.

Обратный проход. *Подстройка весов выходного слоя.* Так как для каждого нейрона выходного слоя задано целевое значение, то подстройка весов легко осуществляется с использованием модифицированного дельта-правила из гл. 2. Внутренние слои называют «скрытыми слоями», для их выходов не имеется целевых значений для сравнения. Поэтому обучение усложняется.

На рис. 3.5 показан процесс обучения для одного веса от нейрона p в скрытом слое j к нейрону q в выходном слое k . Выход нейрона слоя k , вычитаясь из целевого значения (Target), дает сигнал ошибки. Он умножается на производную сжимающей функции $[\text{OUT}(1 - \text{OUT})]$, вычисленную для этого нейрона слоя k , давая, таким образом, величину δ .

$$\delta = \text{OUT}(1 - \text{OUT})(\text{Target} - \text{OUT}) \quad (3.4)$$

Затем δ умножается на величину OUT нейрона j , из которого выходит рассматриваемый вес. Это произведение в свою очередь умножается на коэффициент скорости обучения η (обычно от 0,01 до 1,0), и результат прибавляется к весу. Такая же процедура выполняется для каждого веса от нейрона скрытого слоя к нейрону в выходном слое.

Следующие уравнения иллюстрируют это вычисление:

$$\Delta w_{pq,k} = \eta \delta_{q,k} \text{OUT} \quad (3.5)$$

$$w_{pq,k}(n+1) = w_{pq,k}(n) + \Delta w_{pq,k} \quad (3.6)$$

где $w_{pq,k}(n)$ – величина веса от нейрона p в скрытом слое к нейрону q в выходном слое на шаге n (до коррекции); отметим, что индекс k относится к слою, в котором

заканчивается данный вес, т. е., согласно принятому в этой книге соглашению, с которым он объединен; $w_{pq,k}(n+1)$ – величина веса на шаге $n + 1$ (после коррекции); $\delta_{q,k}$ – величина δ для нейрона q , в выходном слое k ; $OUT_{p,j}$ – величина OUT для нейрона p в скрытом слое j .

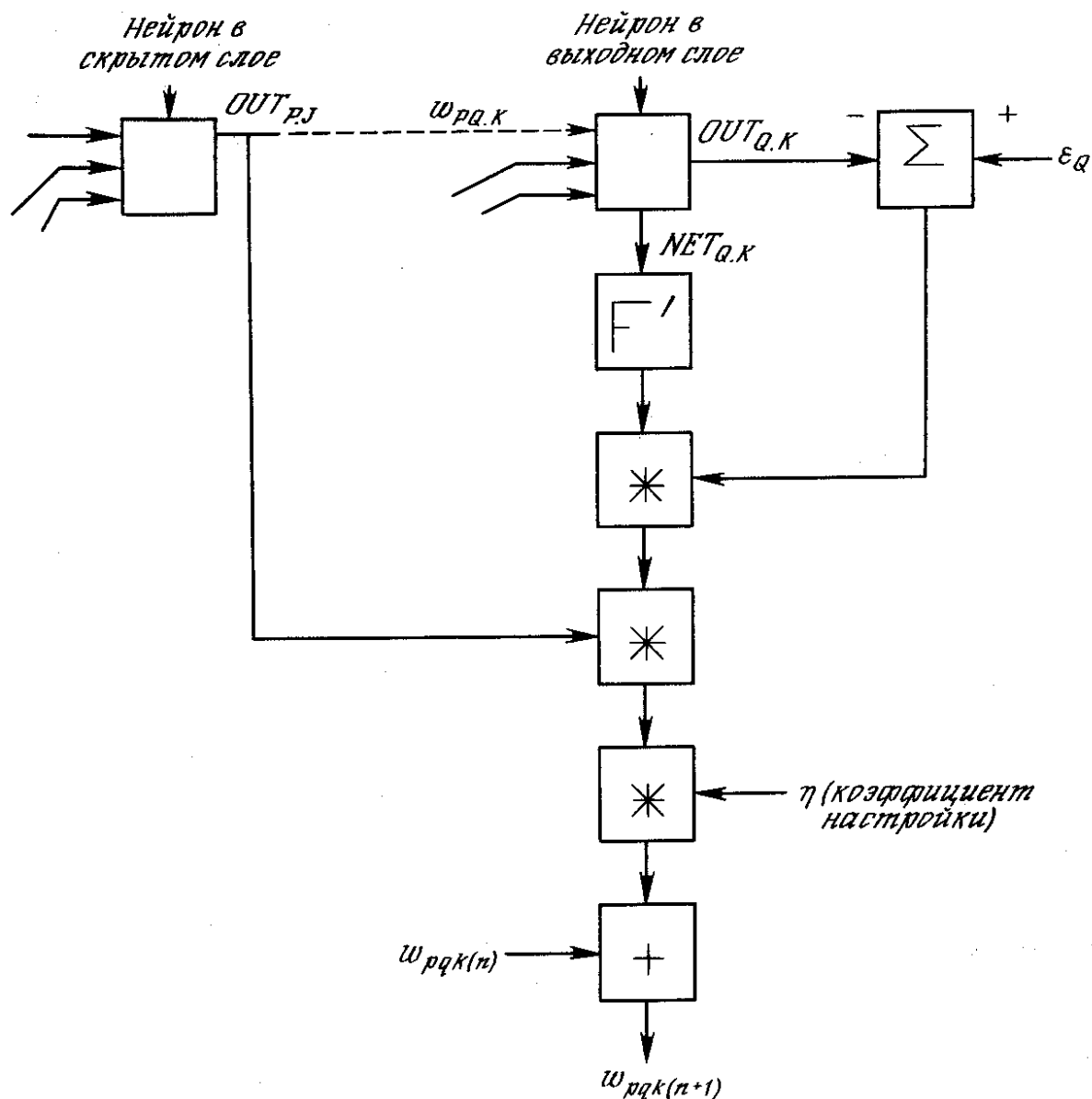


Рис. 3.5. Настройка веса в выходном слое

Подстройка весов скрытого слоя. Рассмотрим один нейрон в скрытом слое, предшествующем выходному слою. При проходе вперед этот нейрон передает свой выходной сигнал нейронам в выходном слое через соединяющие их веса. Во время обучения эти веса функционируют в обратном порядке, пропуская величину δ от выходного слоя назад к скрытому слою. Каждый из этих весов умножается на величину δ нейрона, к которому он присоединен в выходном слое. Величина δ , необходимая для

нейрона скрытого слоя, получается суммированием всех таких произведений и умножением на производную сжимающей функции:

$$\delta_{q,k} = \text{OUT}_{p,j} (1 - \text{OUT}_{p,j}) \left[\sum_q \delta_{q,k} w_{pq,k} \right] \quad (3.7)$$

(см. рис. 3.6). Когда значение δ получено, веса, питающие первый скрытый уровень, могут быть подкорректированы с помощью уравнений (3.5) и (3.6), где индексы модифицируются в соответствии со слоем.

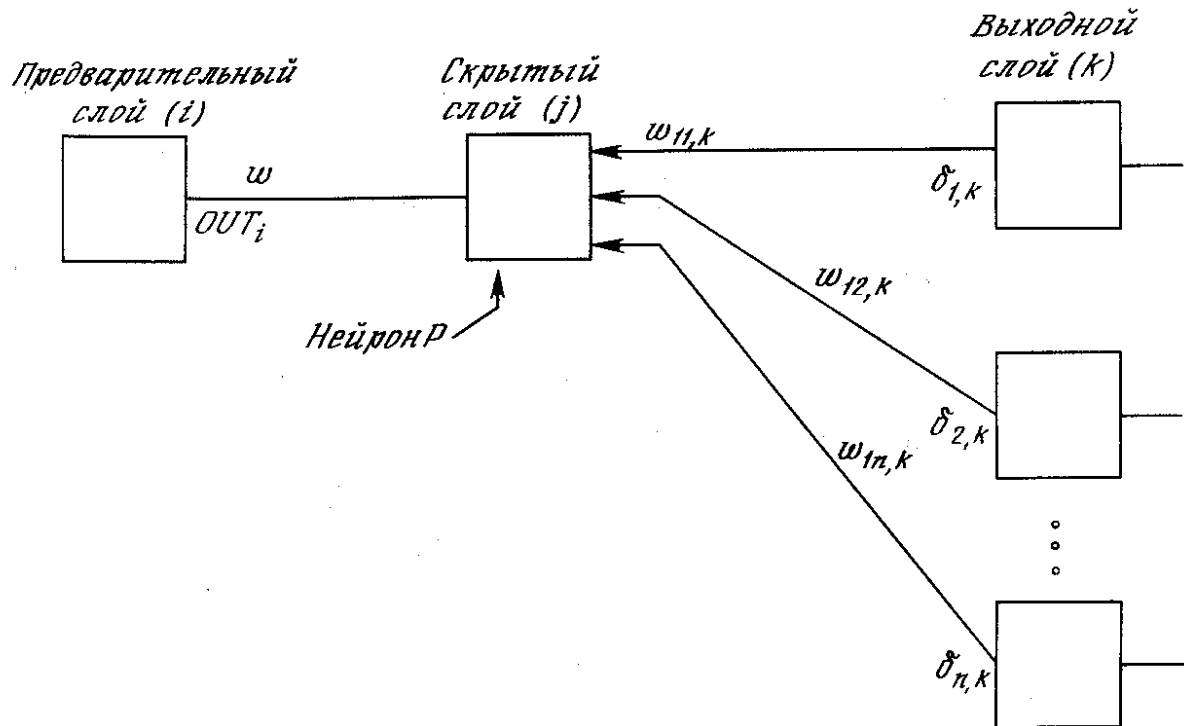


Рис. 3.6. Настройка веса в скрытом слое

Для каждого нейрона в данном скрытом слое должно быть вычислено δ и подстроены все веса, ассоциированные с этим слоем. Этот процесс повторяется слой за слоем по направлению к входу, пока все веса не будут подкорректированы.

С помощью векторных обозначений операция обратного распространения ошибки может быть записана значительно компактнее. Обозначим множество величин δ выходного слоя через $\mathbf{D}_k$ и множество весов выходного слоя как массив $\mathbf{W}_k$. Чтобы получить $\mathbf{D}_j$, δ -вектор выходного слоя, достаточно следующих двух операций:

1. Умножить δ -вектор выходного слоя $\mathbf{D}_k$ на транспонированную матрицу весов $\mathbf{W}_k'$, соединяющую скрытый уровень с выходным уровнем.
2. Умножить каждую компоненту полученного произведения на производную сжимающей функции соответствующего нейрона в скрытом слое.

В символьной записи

$$\mathbf{D}_j = \mathbf{D}_k \mathbf{W}'_k \mathbf{S}[\mathbf{0}_j \mathbf{S}(\mathbf{I} - \mathbf{0}_j)], \quad (3.8)$$

где оператор $\mathbf{S}$ в данной книге обозначает покомпонентное произведение векторов, $\mathbf{0}_j$ – выходной вектор слоя j и $\mathbf{I}$ – вектор, все компоненты которого равны 1.

Добавление нейронного смещения. Во многих случаях желательно наделять каждый нейрон обучаемым смещением. Это позволяет сдвигать начало отсчета логистической функции, давая эффект, аналогичный подстройке порога персептронного нейрона, и приводит к ускорению процесса обучения. Эта возможность может быть легко введена в обучающий алгоритм с помощью добавляемого к каждому нейрону веса, присоединенного к +1. Этот вес обучается так же, как и все остальные веса, за исключением того, что подаваемый на него сигнал всегда равен +1, а не выходу нейрона предыдущего слоя.

Импульс. В работе [7] описан метод ускорения обучения для алгоритма обратного распространения, увеличивающий также устойчивость процесса. Этот метод, названный импульсом, заключается в добавлении к коррекции веса члена, пропорционального величине предыдущего изменения веса. Как только происходит коррекция, она «запоминается» и служит для модификации всех последующих коррекций. Уравнения коррекции модифицируются следующим образом:

$$\Delta w_{pq,k}(n+1) = \eta \delta_{q,k} \text{OUT}_{p,j} + \alpha \Delta w_{pq,k}(n) \quad (3.9)$$

$$w_{pq,k}(n+1) = w_{pq,k}(n) + \Delta w_{pq,k}(n+1) \quad (3.10)$$

где (α – коэффициент импульса, обычно устанавливается около 0,9.

Используя метод импульса, сеть стремится идти по дну узких оврагов поверхности ошибки (если таковые имеются), а не двигаться от склона к склону. Этот метод, по-видимому, хорошо работает на некоторых задачах, но дает слабый или даже отрицательный эффект на других.

В работе [8] описан сходный метод, основанный на экспоненциальном сглаживании, который может иметь преимущество в ряде приложений.

$$\Delta w_{pq,k}(n+1) = (1-\alpha) \delta_{q,k} \text{OUT}_{p,j} + \alpha \Delta w_{pq,k}(n) \quad (3.9)$$

Затем вычисляется изменение веса

$$w_{pq,k}(n+1) = w_{pq,k}(n) + \eta \Delta w_{pq,k}(n+1), \quad (3.10)$$

где α коэффициент сглаживания, варьируемый в диапазоне от 0,0 до 1,0. Если α равен 1,0, то новая коррекция игнорируется и повторяется предыдущая. В области между 0 и 1 коррекция веса сглаживается величиной, пропорциональной α . По-прежнему, η является коэффициентом скорости обучения, служащим для управления средней величиной изменения веса.

ДАЛЬНЕЙШИЕ АЛГОРИТМИЧЕСКИЕ РАЗРАБОТКИ

Многими исследователями были предложены улучшения и обобщения описанного выше основного алгоритма обратного распространения. Литература в этой области слишком обширна, чтобы ее можно было здесь охватить. Кроме того, сейчас еще слишком рано давать окончательные оценки. Некоторые из этих подходов могут оказаться действительно фундаментальными, другие же со временем исчезнут. Некоторые из наиболее многообещающих разработок обсуждаются в этом разделе.

В [5] описан метод ускорения сходимости алгоритма обратного распространения. Названный обратным распространением второго порядка, он использует вторые производные для более точной оценки требуемой коррекции весов. В [5] показано, что этот алгоритм оптимален в том смысле, что невозможно улучшить оценку, используя производные более высокого порядка. Метод требует дополнительных вычислений по сравнению с обратным распространением первого порядка, и необходимы дальнейшие эксперименты для доказательства оправданности этих затрат.

В [9] описан привлекательный метод улучшения характеристик обучения сетей обратного распространения. В работе указывается, что общепринятый от 0 до 1 динамический диапазон входов и выходов скрытых нейронов неоптимален. Так как величина коррекции веса $\Delta w_{pq,k}$ пропорциональна выходному уровню нейрона, порождающего $OUT_{p,j}$, то нулевой уровень ведет к тому, что вес не меняется. При двоичных входных векторах половина входов в среднем будет равна нулю, и веса, с которыми они связаны, не будут обучаться! Решение состоит в приведении входов к значениям $\pm 1/2$ и добавлении смещения к сжимающей функции, чтобы она также принимала значения $\pm 1/2$. Новая сжимающая функция выглядит следующим образом:

$$OUT = -\frac{1}{2} + \frac{1}{1 + e^{-NET}}. \quad (3.13)$$

С помощью таких простых средств время сходимости сокращается в среднем от 30 до 50%. Это является одним из примеров практической модификации, существенно улучшающей характеристику алгоритма.

В [6] и [1] описана методика применения обратного распространения к сетям с обратными связями, т. е. к таким сетям, у которых выходы подаются через обратную связь на входы. Как показано в этих работах, обучение в подобных системах может быть очень быстрым и критерии устойчивости легко удовлетворяются.

ПРИМЕНЕНИЯ

Обратное распространение было использовано в широкой сфере прикладных исследований. Некоторые из них описываются здесь, чтобы продемонстрировать мощь этого метода.

Фирма NEC в Японии объявила недавно, что обратное распространение было ею использовано для визуального распознавания букв, причем точность превысила 99%. Это улучшение было достигнуто с помощью комбинации обычных алгоритмов с сетью обратного распространения, обеспечивающей дополнительную проверку.

В работе [8] достигнут впечатляющий успех с Net-Talk, системой, которая превращает печатный английский текст в высококачественную речь. Магнитофонная запись процесса обучения сильно напоминает звуки ребенка на разных этапах обучения речи.

В [2] обратное распространение использовалось в машинном распознавании рукописных английских слов. Буквы, нормализованные по размеру, наносились на сетку, и брались проекции линий, пересекающих квадраты сетки. Эти проекции служили затем входами для сети обратного распространения. Сообщалось о точности 99,7% при использовании словарного фильтра.

В [3] сообщалось об успешном применении обратного распространения к сжатию изображений, когда образы представлялись одним битом на пиксель, что было восьмикратным улучшением по сравнению с входными данными.

ПРЕДОСТЕРЕЖЕНИЕ

Несмотря на многочисленные успешные применения обратного распространения, оно не является панацеей. Больше всего неприятностей приносит неопределенно долгий процесс обучения. В сложных задачах для обучения сети могут потребоваться дни или даже недели, она может и вообще не обучиться. Длительное время обучения может быть результатом неоптимального выбора длины шага. Неудачи в обучении обычно возникают по двум причинам: паралича сети и попадания в локальный минимум.

Паралич сети

В процессе обучения сети значения весов могут в результате коррекции стать очень большими величинами. Это может привести к тому, что все или большинство нейронов будут функционировать при очень больших значениях OUT, в области, где производная сжимающей функции очень мала. Так как посылаемая обратно в процессе обучения ошибка пропорциональна этой производной, то процесс обучения может

практически замереть. В теоретическом отношении эта проблема плохо изучена. Обычно этого избегают уменьшением размера шага η , но это увеличивает время обучения. Различные эвристики использовались для предохранения от паралича или для восстановления после него, но пока что они могут рассматриваться лишь как экспериментальные.

Локальные минимумы

Обратное распространение использует разновидность градиентного спуска, т. е. осуществляет спуск вниз по поверхности ошибки, непрерывно подстраивая веса в направлении к минимуму. Поверхность ошибки сложной сети сильно изрезана и состоит из холмов, долин, складок и оврагов в пространстве высокой размерности. Сеть может попасть в локальный минимум (неглубокую долину), когда рядом имеется гораздо более глубокий минимум. В точке локального минимума все направления ведут вверх, и сеть неспособна из него выбраться. Статистические методы обучения могут помочь избежать этой ловушки, но они медленны. В [10] предложен метод, объединяющий статистические методы машины Коши с градиентным спуском обратного распространения и приводящий к системе, которая находит глобальный минимум, сохраняя высокую скорость обратного распространения. Это обсуждается в гл. 5.

Размер шага

Внимательный разбор доказательства сходимости в [7] показывает, что коррекции весов предполагаются бесконечно малыми. Ясно, что это неосуществимо на практике, так как ведет к бесконечному времени обучения. Размер шага должен браться конечным, и в этом вопросе приходится опираться только на опыт. Если размер шага очень мал, то сходимость слишком медленная, если же очень велик, то может возникнуть паралич или постоянная неустойчивость. В [11] описан адаптивный алгоритм выбора шага, автоматически корректирующий размер шага в процессе обучения.

Временная неустойчивость

Если сеть учится распознавать буквы, то нет смысла учить «Б», если при этом забывается «А». Процесс обучения должен быть таким, чтобы сеть обучалась на всем обучающем множестве без пропусков того, что уже выучено. В доказательстве сходимости [7] это условие выполнено, но требуется также, чтобы сети предъявлялись все векторы обучающего множества прежде, чем выполняется коррекция весов. Необходимые изменения весов должны вычисляться на всем множестве, а это требует дополнительной памяти; после ряда таких обучающих циклов веса сойдутся к

минимальной ошибке. Этот метод может оказаться бесполезным, если сеть находится в постоянно меняющейся внешней среде, так что второй раз один и тот же вектор может уже не повториться. В этом случае процесс обучения может никогда не сойтись, бесцельно блуждая или сильно осциллируя. В этом смысле обратное распространение не похоже на биологические системы.

Контрольные вопросы

1. Что такое процедура обратного распространения и почему она важна для обучения многослойных нейронных сетей?
2. Какие основные этапы включает в себя обучающий алгоритм обратного распространения?
3. Почему активационные функции типа сигмоида часто применяются в нейронных сетях и какова их роль в процедуре обратного распространения?
4. Объясните механизм работы многослойной нейронной сети с точки зрения прохождения сигнала («прямой проход»).
5. Как настраиваются веса в скрытых слоях нейронной сети? Объясните подробно процесс настройки и формулу, используемую для обновления весов.
6. Перечислите известные проблемы и недостатки, возникающие при обучении нейронных сетей методом обратного распространения (например, локальные минимумы, временная нестабильность, масштабирование входных данных).